

SolarSystem3D

Aurora Di Stasi 6455527

Nel progetto ho realizzato una rappresentazione tridimensionale del Sistema Solare, dove è possibile osservare il moto di rotazione dei pianeti su sé stessi e il moto di rivoluzione intorno al Sole, i principali satelliti e l'anello di Saturno. La camera si può ruotare, avvicinare, allontanare e agganciare ad un corpo per seguirlo mentre si muove. La simulazione, inoltre, si può mettere in pausa, accelerare o rallentare.

Il progetto è organizzato in 10 tappe, dove ogni tappa è una cartella autonoma che aggiunge una funzionalità rispetto alla precedente, fino ad arrivare all'ultima che contiene il simulatore completo.

Tappa01

In questa tappa ho generato una finestra SFML e creato un contesto OpenGL 4.1 con la classe Setup caricando le funzioni della libreria con GLAD.

La classe mesh genera una sfera dividendola in settori e pile e calcola la normale per ogni vertice. Il colore viene calcolato dal fragment shader con il modello di Lambert.

In questa tappa non ci sono ancora trasformazioni.

Tappa02

Nella seconda tappa ho introdotto le trasformazioni geometriche e la gestione della camera. Ho creato una struttura CorpoCeleste che descrive un corpo con il suo nome, raggio, distanza orbitale, velocità di rivoluzione e rotazione, inclinazione dell'asse e colore. Con il metodo model(t) vengono combinate 4 trasformazioni per portare la sfera nel mondo nell'istante t.

Nella tappa precedente la direzione della luce era una costante, ora invece viene calcolata per ogni frammento come la direzione dal punto della superficie verso il sole (che si trova nell'origine).

La scena contiene solo il Sole che è la sorgente della luce e quindi non viene illuminato, ma la struttura è già completa per aggiungere altri corpi poi nelle tappe successive.

Tappa03

In questa tappa ho completato la scena con gli 8 pianeti e ho aggiunto il disegno delle orbite. La classe `Orbita` genera un cerchio con 128 vertici che vengono collegati e chiusi da `GL_LINE_LOOP`. Questa geometria viene usata per tutte le orbite e scalata dalla matrice di modellazione in base al raggio dell'orbita di ogni pianeta. Prima avevo solo 2 categorie da disegnare (pianeti e sole) e per questo avevo la uniform booleana `isSun`, in questa tappa è stata sostituita da una uniform intera tipo, per selezionare il ramo da eseguire nel fragment shader.

Ho migliorato la camera perché nella tappa precedente la matrice di proiezione usava lo stesso valore sull'asse orizzontale e verticale. Il volume di vista canonico però è un cubo e la finestra è rettangolare, per questo la viewport deformava l'immagine e le sfere apparivano schiacciate. Ho quindi introdotto il parametro `aspect = width / height`, così la componente orizzontale della matrice diventa `focal_distance / aspect` e la compressione introdotta dalla proiezione annulla lo stiramento della viewport.

Il metodo `set_aspect` viene richiamato a ogni ridimensionamento della finestra, perché `width` e `height` cambiano.

Tappa04

In questa tappa l'utente può controllare la scena, ho introdotto un tempo di simulazione diverso da quello reale e i comandi da tastiera.

L'utente può agganciare la camera ad un corpo, mettere in pausa, accelerare o rallentare il tempo, ripristinare la velocità e mostrare o nascondere le orbite.

Il tempo di simulazione aumenta di una quantità pari al tempo realmente trascorso moltiplicato per un fattore di velocità e resta fermo quando la simulazione è in pausa.

Le posizioni dei corpi dipendono dal tempo di simulazione.

Tappa05

Nella tappa 5 ho generato il cielo stellato nel fragment shader.

La sfera del cielo è la stessa mesh dei pianeti con raggio 150 e centrata nell'origine. La camera si trova al suo interno e vede la faccia interna.

Ho dovuto aggiungere un tetto allo zoom a 140, quindi inferiore al raggio della sfera cielo, perché quando mi allontanavo oltre la camera usciva dalla sfera del cielo.

Tappa06

In questa tappa ho aggiunto le texture ai corpi celesti.

Ho aggiunto alla mesh come terzo attributo le coordinate texture (u,v).

La funzione carica_texture legge l'immagine, la trasferisce in memoria video con `glTexImage2D` e genera le mipmap. L'identificativo delle texture viene inoltre memorizzato nella struttura del `CorpoCeleste`.

Nel fragment shader il colore dei pianeti non viene più dalla uniform `objectColor` ma dalla texture moltiplicata per il fattore di illuminazione diffusa. Anche il Sole usa la propria texture, ma senza illuminazione, perché è la sorgente della luce.

Tappa07

Nella tappa 7 ho aggiunto la Luna, quindi un corpo che non orbita attorno al sole ma attorno ad un altro corpo che si muove. Per farlo ho aggiunto alla struttura `CorpoCeleste` un campo genitore, che contiene l'indice del corpo attorno a cui orbita il satellite (-1 per corpi che orbitano attorno al Sole).

`Trasformazione_corpo` calcola la matrice di modellazione del corpo e se ha un genitore la moltiplica per la traslazione orbitale del genitore, quindi dal genitore viene ereditata solo la traslazione, per questo l'ho estratta da `model` e creato un metodo a parte chiamato `orbita`.

Ho scelto di non disegnare le orbite dei satelliti perché sennò si intersecavano all'orbita del pianeta genitore.

Infine, in questa tappa è emerso che il raggio orbitale dei satelliti avrebbe intersecato i pianeti vicini, il problema è stato risolto nella tappa 8.

Tappa08

Ho aggiunto altri satelliti: Io, Europa, Ganimede e Callisto attorno a Giove, Encelado e Titano attorno a Saturno, Tritone attorno a Nettuno. Tritone ha velocità di rivoluzione e di rotazione negative, perché la sua orbita è retrograda.

Con il raggiungimento di 17 corpi celesti, dovevo trovare un modo per selezionarli visto che le cifre non bastavano, quindi ho assegnato alle cifre i pianeti e attraverso il tasto S la possibilità di scorrere i satelliti del corpo genitore.

In questa tappa ho dovuto ricalcolare le distanze dal Sole dei pianeti.

Tappa09

In questa tappa ho aggiunto l'anello di Saturno con la classe anello che genera due cerchi con all'interno una fila di triangoli.

Il ciclo compie un passo in più dei 128 settori (per richiudere il giro) e a ogni passo aggiunge un vertice sul cerchio interno e uno su quello esterno, entrambi sullo stesso raggio. poi il disegno usa `GL_TRIANGLE_STRIP`, in cui ogni vertice nuovo forma un triangolo con i due precedenti.

Per la posizione dell'anello ho estratto da model anche l'inclinazione assiale e creato un metodo `tilt`, perché doveva ereditarla da Saturno.

Tappa10

In quest'ultima tappa ho completato il modello di illuminazione aggiungendo il termine speculare, seguendo il modello Blinn-Phong. Fino alla tappa precedente il colore di un pianeta era dato dalla componente ambiente più quella diffusa di Lambert. Ora il termine speculare descrive la luce del Sole che rimbalza sulla superficie e arriva all'osservatore. Per questo ho calcolato l'inversa della matrice di vista per sapere dove si trova la camera nel sistema del mondo.

Ogni corpo ha 2 parametri in più: il coefficiente di riflessione speculare e l'esponente p . Questo permette di attribuire ad ogni pianeta caratteristiche aggiuntive, perché così è possibile definire come ognuno riflette la luce. Per esempio: Ganimede e Callisto hanno superfici ghiacciate, ma molto antiche e ricoperte da materiale polveroso, quindi vengono trattate come superfici ruvide/diffuse. Europa ed Encelado, invece, hanno superfici di ghiaccio molto più pulite e riflettenti e quindi una componente speculare più elevata.

Scelta delle dimensioni, delle distanze e delle velocità

Per risolvere i problemi di scala, per quanto riguarda la dimensione dei corpi ho pensato di dividerli in 4 famiglie, quindi stella, pianeti gassosi, ghiacciati e

rocciosi. Così in ogni famiglia i rapporti sono esatti e per ogni famiglia applico un fattore di riduzione. Il fattore del Sole è fissato dal vincolo che il Sole non deve invadere l'orbita di Mercurio.

Per le distanze dal sole ho rapportato in base alla distanza della terra e attenuato rispetto ai valori reali, per esempio Nettuno è distante 30 volte la Terra e nel modello lo è circa 4 volte.

Nella tappa 8 sono poi state ricalcolate le distanze, per impedire ai satelliti di entrare nell'orbita del pianeta precedente e successivo.

Per la velocità ho scelto quanti secondi deve durare un giro completo, per esempio la Terra, che nella realtà impiega un anno per la rivoluzione, nel modello ci mette 20 secondi e gli altri pianeti saranno per lo più proporzionati a questi secondi applicando però anche qui delle attenuazioni quando è necessario.

Quindi ogni corpo più grande, lontano o veloce nella realtà lo è anche nel modello.

Inclinazioni assiali, verso di rotazione e verso di rivoluzione sono valori reali.

Le orbite le ho trattate come circolari e non come ellittiche. Questa scelta l'ho presa perché visto che l'eccentricità delle orbite è molto piccola, la differenza visiva sarebbe stata minima e avrebbe aumentato la complessità.

Codice esterno, risorse e strumenti utilizzati

Dai laboratori del corso ho preso:

- `hotshaders.hh`
- `gl.h`

Algoritmi presi da documentazione e tutorial:

- www.songho.ca/opengl/gl_sphere.html -> usato come supporto per metodo `build_sphere`

Texture:

- <https://www.deviantart.com/search?q=texture+planet>
- <https://www.solarsystemscope.com/textures/>

Il cielo stellato procedurale nel fragment shader è stato scritto con il supporto di Claude.